

HarborSync Mikroservis Analizi ve Multi-Master Kubernetes Tasarımı

İçindekiler

1. Yönetici Özeti
2. Kaynaklardan Dogrulanen Bulgular
3. Uygulama Mimarisi
 - 3.1 Hiyerarsik Diyagram
 - 3.2 Event/Dependency Diyagrami
 - 3.3 Senkron ve Asenkron Bagimliliklar
4. Servis Sayisi
5. Stateless / Stateful Ayrimi
6. Trafik Beklentisi ve Yogun Servisler
7. Database ve Bagimliliklar
 - 7.1 PostgreSQL
 - 7.2 Redis
 - 7.3 RabbitMQ
 - 7.4 Diger Bagimliliklar
8. CI/CD Ihtiyaci
9. HarborSync Icin Multi-Master Kubernetes Tasarimi
 - 9.1 Tasarim Varsayimlari
 - 9.2 HarborSync Servislerinin Kubernetes Karsiligi
 - 9.3 Hedef Multi-Master Topoloji
 - 9.4 Cluster Node Diyagrami
 - 9.5 Uygulama Kaynak Hesabi
 - 9.6 Önerilen VM Kaynaklari
 - 9.7 Ortam ve Namespace Plani
 - 9.8 Add-on Plani
 - 9.9 Ingress, Service ve Network Akisi
 - 9.10 Scheduling, Anti-Affinity ve PDB
 - 9.11 HarborSync Icin Karar Özeti
10. HA Ihtiyaci
11. Backup/Restore Ihtiyaci
12. Güvenlik Ihtiyaci
13. CPU/RAM Ihtiyaci
 - 13.1 Uygulama Pod Baslangiç Kaynaklari
 - 13.2 Stateful Altyapi Baslangiç Kaynaklari
 - 13.3 Çalıştırma Bazlı Kaynak Ölçümü ve Net Request/Limit Tablosu
 - 13.4 Cluster Kapasite Önerisi
14. Replica Ihtiyaci
15. Operasyonel Gözlemler ve İyileştirme Listesi
16. Sonuç
17. Proxmox Ihtiyaci - Demo Ortami
 - 17.1 Demo Varsayimi

- 17.2 Istenen Demo VM Kaynaklari
- 17.3 Minimum Demo Toplami
- 17.4 Rahat Demo Toplami
- 17.5 Proxmox Fiziksel Host Önerisi
- 17.6 Demo Icin Kaynaklari Küçük Tutma Kararlari
- 17.7 Net Talep Metni
- 18. Uygulama Fazi - Güncel Mentor Kararina Göre Yol Haritasi
 - 18.1 Bu Projede Liderlik Prensibi
 - 18.2 Güncel Hedef Mimari
 - 18.3 Node Roller
 - 18.4 Uygulama Sirasi
 - 18.5 Kubernetes İçindeki Stateful Bilesenler
 - 18.6 Namespace Plani
 - 18.7 Manifest ve Helm Yaklaşımı
 - 18.8 HarborSync Deploy Sirasi
 - 18.9 İlk Basit Basari Kriteri
 - 18.10 Senden Beklediğim Çalışma Şekli
 - 18.11 Bu Fazda Ertelediklerimiz
 - 18.12 Benim Sana Lead Edeceğim Uygulama Sirasi

Bu rapor, `/home/deniz.yaldiz/practice_k8ns/HarborSync` altındaki kaynak kod, Docker Compose, servis konfigürasyonlari, event sözleşmeleri, testler ve Postman/Newman artifact'lari incelenerek hazirlandi. Kodda dogrulanen bilgiler ayrıca belirtildi; kapasite ve Kubernetes önerileri ise mevcut demo yükü üzerine üretim hazirligi varsayimidir.

1. Yönetici Özeti

HarborSync, liman sahasından gelen drone telemetrisini alıp konteyner sektörü doluluk/tikaniklik durumunu analiz eden, gerektiğinde görev üreten ve olaylari bildirim loglarına aktaran polyglot bir mikroservis sistemidir.

Koddan dogrulanana ana uygulama servis sayisi 6'dır:

#	Servis	Teknoloji	Port	Rol
1	API Gateway	Spring Cloud Gateway	8080	Giris kapisi, JWT, correlation ID, Redis rate limit
2	Vessel Service	Spring Boot 3 + PostgreSQL	8081	Gemi kaydi/status/berth yönetimi, lifecycle event üretimi
3	Telemetry Service	Go 1.22 + Redis + RabbitMQ	8082	Drone telemetri ingest, Redis son durum, telemetry.processed üretimi
4	Congestion Analysis	Spring Boot 3 + RabbitMQ	8083	Telemetry tüketimi, tikaniklik kural motoru, alert üretimi
5	Task Assignment	Spring Boot 3 + PostgreSQL + RabbitMQ	8084	Alert tüketimi, görev üretimi, Vessel Service senkron çağri
6	Notification Service	FastAPI + RabbitMQ	8085	Event tüketimi, rotating structured log

Altyapi bilesenleri uygulama servis sayisina dahil degildir: RabbitMQ, PostgreSQL-vessel, PostgreSQL-task, Redis. Drone Simulator yardimci trafik üretici script'tir, platform runtime mikroservisi olarak sayilmamalidir.

Mevcut proje Docker Compose odaklidir; Kubernetes/Helm manifesti ve CI/CD pipeline dosyasi yoktur. Multi-master Kubernetes tasarimi için öneri: 3 control-plane + 3 worker node, ingress/LB ile Gateway üzerinden giris, PostgreSQL/RabbitMQ/Redis için HA operator veya tercihen managed servis, uygulama pod'lari stateless deployment olarak en az 2 replica.

2. Kaynaklardan Dogrulanen Bulgular

Ana runtime tanimi `docker-compose.yml` içinde yer alır. RabbitMQ, iki ayrı PostgreSQL instance'i, Redis ve 6 uygulama servisi burada tanımlıdır (`docker-compose.yml:1-171`). Gateway route ve rate-limit ayarlari `gateway/src/main/resources/application.yml:11-37` içinde; JWT secret `gateway/src/main/resources/application.yml:48-49` ile verilir.

Telemetry Service `POST /telemetry/ingest` ve `GET /health` endpointlerini açar (`telemetry-service/main.go:132-135`). Ingest akisi payload'i valide eder, correlation ID üretir veya devralır, fill rate hesaplar, Redis'e drone state yazar ve RabbitMQ'ya event basar (`telemetry-service/main.go:145-183`). Redis yazma hatasi non-fatal ele alınmıştır; RabbitMQ publish hatasi HTTP 502 döndürür.

Congestion Analysis fill rate ve blockage kurallarini uygular: blockage + ETA varsa `IMMEDIATE_ACTION`, kritik esik üzeri `SECTOR_CRITICAL`, warning esigi üzeri `SECTOR_WARNING` (`congestion-analysis/.../CongestionRuleEngine.java:28-46`).

Task Assignment alert geldiginde Vessel Service'ten arriving vessel listesini çeker, mümkünse berth reserve eder, task tablosuna yazar ve `task.created` event'i üretir (`task-assignment-service/.../TaskAssignmentService.java:47-92`). Publish hatasinda task'i FAILED yapar ve mümkünse berth release ile kompanzasyon dener (`TaskAssignmentService.java:93-122`).

Notification Service RabbitMQ'dan congestion, task ve vessel lifecycle queue'larini tüketir; consumer QoS prefetch 10 olarak ayarlanmıştır (`notification-service/consumer.py:195-245`). Loglar dosya ve console'a yazilir.

Postman/Newman artifact'larina göre 2026-06-11 tarihinde endpoint koleksiyonu 21 request/23 assertion ve simulation koleksiyonu 10 request/10 assertion ile hatasiz kosmustur; ortalama response süreleri sirasiyla 15 ms ve 14 ms olarak kaydedilmiştir (`postman/run-artifacts/newman-run-summary.md:21-58`).

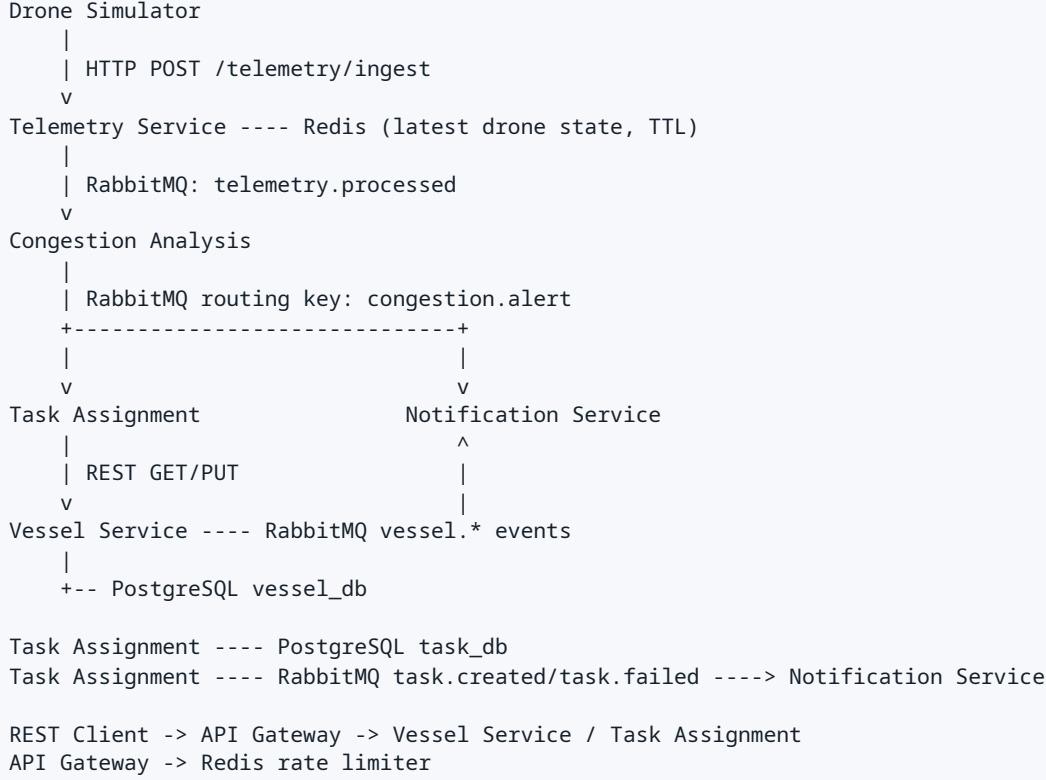
3. Uygulama Mimarisi

3.1 Hiyerarsik Diyagram

```
HarborSync Platform
|
+-- External Clients
|   +-- REST/Postman/Operator Client
|   +-- Drone Simulator
|
+-- Edge Layer
|   +-- API Gateway (8080)
|       +-- JWT signature validation
|       +-- X-Correlation-ID injection
|       +-- Redis based rate limiting
|       +-- Routes:
|           +-- /api/vessels/** -> Vessel Service
|           +-- /api/tasks/**  -> Task Assignment
|
+-- Domain Services
|   +-- Vessel Service (8081)
|       |   +-- REST CRUD/status/berth
|       |   +-- PostgreSQL vessel_db
|       |   +-- RabbitMQ lifecycle producer
|       |
|   +-- Telemetry Service (8082)
|       |   +-- REST telemetry ingest
|       |   +-- Redis latest drone state, TTL 30s
|       |   +-- RabbitMQ telemetry.processed producer
|       |
|   +-- Congestion Analysis (8083)
|       |   +-- RabbitMQ telemetry.processed consumer
|       |   +-- Rule engine
|       |   +-- RabbitMQ congestion.alert producer
|       |
|   +-- Task Assignment (8084)
|       |   +-- RabbitMQ congestion.alert.task-assignment consumer
|       |   +-- REST /tasks API
|       |   +-- PostgreSQL task_db
|       |   +-- Sync dependency: Vessel Service
|       |   +-- RabbitMQ task.created/task.failed producer
|       |
|   +-- Notification Service (8085)
|       +-- RabbitMQ event consumers
|       +-- Rotating file/console structured logs
|
+-- Shared Infrastructure
    +-- RabbitMQ direct exchange: harborsync.exchange
    +-- PostgreSQL vessel_db
    +-- PostgreSQL task_db
    +-- Redis
```

Şekil 1: HarborSync hiyerarsik uygulama mimarisi.

3.2 Event/Dependency Diyagrami



Şekil 2: Servisler arası event ve dependency akisi.

3.3 Senkron ve Asenkron Bagimliliklar

Kaynak	Hedef	Tip	Etki
REST Client	API Gateway	HTTP	Dis dünyaya açılan ana giris
API Gateway	Vessel Service	HTTP	Vessel API proxy
API Gateway	Task Assignment	HTTP	Task API proxy
API Gateway	Redis	TCP	Rate limit state; Redis kesilirse Gateway davranisi etkilenir
Drone Simulator	Telemetry Service	HTTP	Telemetry girisi
Telemetry Service	Redis	TCP	Son drone state, non-critical cache
Telemetry Service	RabbitMQ	AMQP	Telemetry event üretimi; publish hatasi ingest'i basarisiz yapar
Congestion Analysis	RabbitMQ	AMQP	Telemetry tüketimi ve alert üretimi
Task Assignment	RabbitMQ	AMQP	Alert tüketimi, task event üretimi
Task Assignment	Vessel Service	HTTP	Alert isleme sirasinda senkron liste/reserve/release
Vessel Service	PostgreSQL	JDBC	Stateful kalici veri
Task Assignment	PostgreSQL	JDBC	Stateful kalici veri
Notification Service	RabbitMQ	AMQP	Event tüketimi ve DLQ izleme

4. Servis Sayisi

Eksiksiz ayrim:

Kategori	Adet	Bilesenler
Uygulama mikroservisi	6	gateway, vessel-service, telemetry-service, congestion-analysis, task-assignment-service, notification-service
Altyapi/runtime bagimliliği	4 logical	RabbitMQ, Redis, PostgreSQL vessel_db, PostgreSQL task_db
Yardimci tool/script	1	drone-simulator
Test/verification artifact	-	Postman collections, Newman reports, unit tests

Not: PostgreSQL iki ayri container olarak çalisiyor; domain açisindan "database per service" yaklasimi vardir. Kubernetes tasariminda bunlar ayri PostgreSQL cluster/database olarak ele alinmalidir.

5. Stateless / Stateful Ayrımı

Bileşen	Stateless/Stateful	Gerekçe	Kubernetes sonucu
API Gateway	Stateless uygulama; Redis'e bağımlı	Pod içinde kalıcı veri yok, rate limit Redis'te	Deployment, HPA uygun; Redis HA olmalı
Vessel Service	Uygulama stateless, veri stateful	Kalıcı veri PostgreSQL vessel_db'de	Deployment + PostgreSQL HA
Telemetry Service	Uygulama stateless, Redis cache state	Son drone state Redis TTL 30s	Deployment + Redis HA; cache kaybı tolere edilebilir
Congestion Analysis	Stateless	Sadece event tüketir, kural uygular	Deployment, queue consumer scaling
Task Assignment	Uygulama stateless, veri stateful	Kalıcı task verisi PostgreSQL task_db'de	Deployment + PostgreSQL HA
Notification Service	Fiilen stateful log dosyası var	Rotating file log container filesystem'e yazılıyor	Production'da stdout + log aggregation önerilir; dosya PV istenirse StatefulSet gerekebilir ama önerilmez
RabbitMQ	Stateful	Queue, exchange, durable message	RabbitMQ cluster/operator + PV
PostgreSQL	Stateful	Kalıcı relational veri	HA cluster + PV + backup
Redis	Duruma göre	Rate limit ve drone state; compose'da persistence kapalı	HA Redis/Sentinel veya managed; cache-only kabul edilebilir

6. Trafik Beklentisi ve Yogun Servisler

Koddan dogrulanan demo yükü: Drone Simulator varsayilan olarak 2 saniyede bir telemetry istegi atar, 3 drone ve 4 sektör ile rastgele payload üretir (`drone-simulator/simulate.py:13-35`). Gateway her route için IP basina 10 req/s replenish ve 20 burst limiti uygular (`gateway/src/main/resources/application.yml:20-37`).

Üretim varsayimi için üç trafik profili:

Profil	Telemetry girisi	REST API	Event etkisi
Demo/lab	0.5 req/s	Düşük	Tek node Docker Compose yeterli
Pilot liman	50-150 telemetry req/s	10-30 req/s	RabbitMQ ve Congestion yatay ölçek ister
Üretim	300-1000 telemetry req/s	50-200 req/s	RabbitMQ, Telemetry, Congestion ve Task ana kapasite ekseni olur

En yogun olmasi beklenen servisler:

- Telemetry Service: bütün drone verisinin ilk giris noktası; HTTP ingest + Redis write + RabbitMQ publish yapar.
- RabbitMQ: event fan-out ve durable queue yükünün merkezi; HA ve disk I/O kritik.
- Congestion Analysis: telemetry queue tüketim hizini belirler; stateless oldugu için kolay ölçeklenir.
- Task Assignment: alert basina DB write + RabbitMQ publish + Vessel Service HTTP çağrısı yapar; en riskli orkestrasyon noktası.
- PostgreSQL task_db: alert yogunlugunda write-heavy hale gelir.

Olası bottleneckler:

Bottleneck	Neden	Etki	Öneri
RabbitMQ disk/queue birikimi	Durable message + consumer gecikmesi	Uçtan uca gecikme artar	Queue depth alert, quorum queues, consumer HPA
Task Assignment -> Vessel senkron çağrı	Alert isleme içinde 10s bloklayan WebClient call	Consumer throughput düşer	Timeout/circuit breaker mevcut; bulkhead ve async reserve düşünülebilir
PostgreSQL task_db	Her alert task insert yapar	Write latency artar	Index/connection pool tuning, read replica sadece raporlama için
Gateway Redis	Rate limiter Redis'e bagimli	API girisi etkilenir	Redis HA, fail-open/fail-closed kararı
Notification log file	Container diskine log	Pod reschedule'da log kaybi	stdout + Loki/ELK/OpenSearch
Consumer'siz congestion.alert queue	Congestion Analysis kendi queue'sunu declare/bind ediyor ama consume etmiyor (<code>RabbitMQConfig.java:39-66</code>)	RabbitMQ'da gereksiz mesaj birikimi	Bu queue kaldırilmali; sadece routing key ve consumer queue'lari kalmali

7. Database ve Bagimliliklar

7.1 PostgreSQL

Vessel Service:

- DB: `vessel_db`
- Tablo: `vessels`
- Önemli alanlar: `id`, `name`, `imo_number`, `status`, `berth`, `eta`, `timestamps`
- Constraint: IMO unique, status check, status index

Task Assignment:

- DB: `task_db`
- Tablo: `tasks`
- Önemli alanlar: `id`, `sector`, `alert_type`, `assigned_unit`, `priority`, `status`, `correlation_id`, `timestamps`
- Constraint: status/priority check, status ve sector index

7.2 Redis

- Gateway rate limiter backend'i.
- Telemetry latest drone state cache'i: key `drone:<droneId>`, TTL 30s.
- Compose'da Redis persistence kapali: `redis-server --save "" --appendonly no` (`docker-compose.yml:47-52`).

7.3 RabbitMQ

- Exchange: `harborsync.exchange`, `direct`.
- Önemli queue/routing key'ler: `telemetry.processed`, `congestion.alert.task-assignment`, `congestion.alert.notification`, `task.created`, `task.failed`, `vessel.arrived`, `vessel.docked`, `vessel.departed`, `dlq.errors`.
- Business queue'larda DLQ argümanlari kullaniliyor.
- Notification prefetch 10 (`notification-service/consumer.py:195-198`).

7.4 Diger Bagimliliklar

- Java 17, Spring Boot 3.2.5, Spring Cloud Gateway 2023.0.1.
- Go 1.22, `amqp091-go`, `go-redis`.
- Python 3.11, FastAPI, uvicorn, aio-pika.

8. CI/CD İhtiyacı

Mevcut repo içinde `.github`, `.gitlab-ci.yml`, Jenkinsfile, Drone CI veya benzeri pipeline bulunmadi. CI/CD ihtiyacı yüksektir; multi-master Kubernetes hedefi için elle build/deploy sürdürülebilir değildir.

Önerilen pipeline:

```
Pull Request
-> lint/static check
-> unit tests:
    Maven test x4
    go test ./...
    python unittest x2
-> docker build for 6 images
-> container scan + dependency scan + SBOM
-> docker compose config validation
-> optional integration/E2E: compose up + Newman collections
-> push images to registry
-> Helm/Kustomize render validation
-> deploy to dev namespace
-> smoke tests
-> manual approval
-> deploy to stage/prod with rolling/canary
```

Gerekli artifact'lar:

- Her servis için versioned container image.
- Helm chart veya Kustomize overlays: `dev`, `stage`, `prod`.
- Secret yönetimi: External Secrets / Vault / cloud secret manager.
- DB migration job stratejisi: Flyway migration'lari deploy sirasinda kontrollü kosmalı.

9. HarborSync Icin Multi-Master Kubernetes Tasarimi

Bu tasarim, projenin gercek servis yapisi ve ölçülen kaynak tüketimi kullanılarak hazirlandi. Tasarim karari su siraya göre verildi: servis envanteri, stateless/stateful ayrimi, replica sayisi, container request/limit degerleri, add-on ihtiyaci, worker kapasitesi, control-plane HA, load balancer HA, dis bagimliliklar, monitoring/logging ve Proxmox toplam kapasitesi.

9.1 Tasarim Varsayimlari

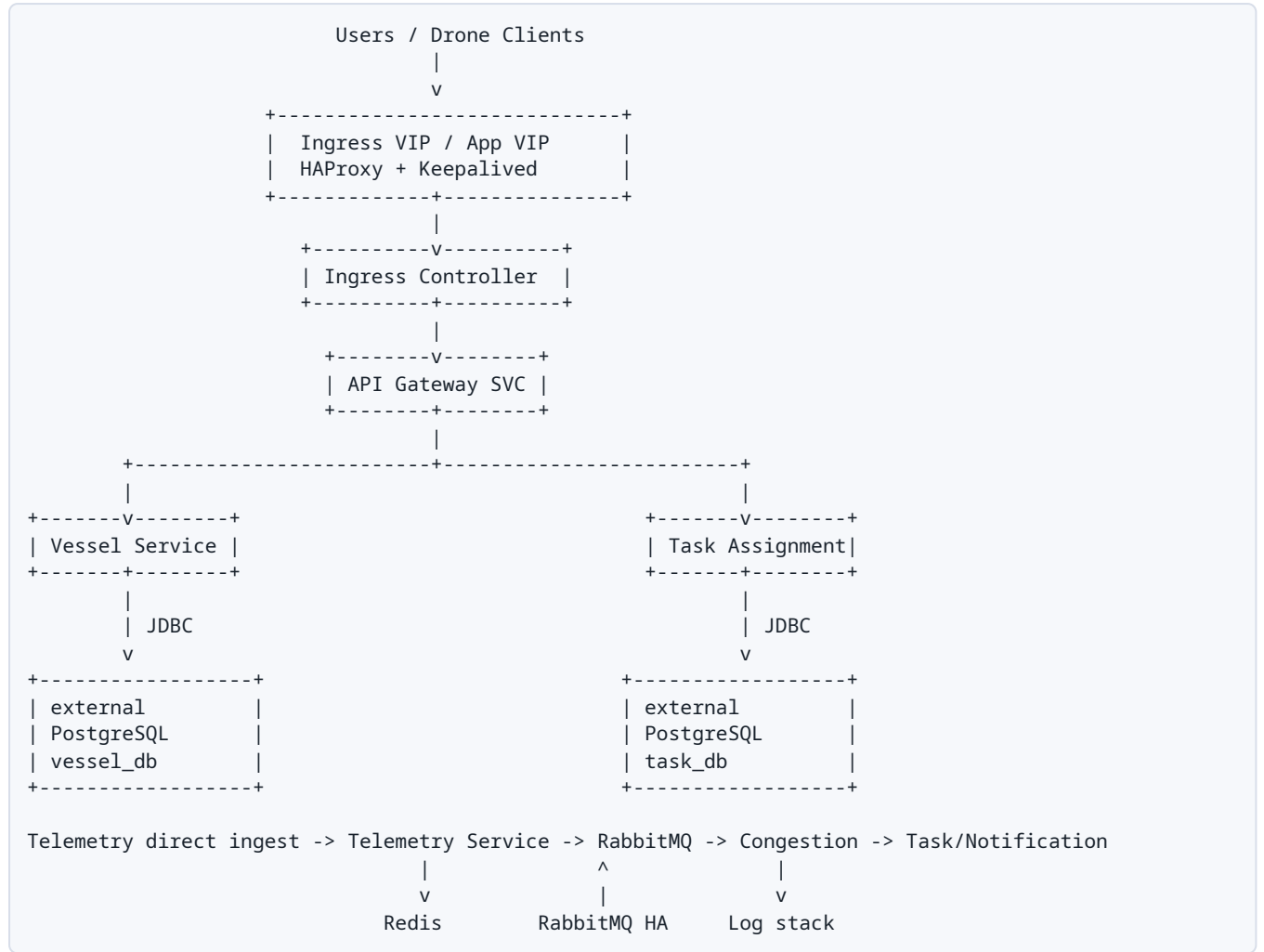
Karar	HarborSync icin secim	Gerekce
Kubernetes tipi	Multi-master kubeadm cluster	Control-plane HA ve DevOps pratigi hedefi
etcd modeli	Stacked etcd	Lab/pilot icin daha sade; master disk latency kritik
Load balancer	2 VM, HAProxy + Keepalived	API server ve Ingress VIP icin HA pratigi
Worker sayisi	3 worker	Pod yayma, rolling update, node failure ve anti-affinity testi
Database	Kubernetes disinda DB VM veya managed DB	Stateful karmasikligi azaltir, DB backup/restore ayrisir
RabbitMQ/Redis	Lab icin cluster ici olabilir; prod icin managed/operator	Mesajlasma ve rate-limit kritik bagimliliklar
Ortam	Once dev, sonra staging, prod simülasyonu en son	Kaynak tüketimini kontrollü büyötmek icin
Monitoring/GitOps	Argo CD + Prometheus + Grafana + Loki önerilir	DevOps pratiği ve görünürlük icin

9.2 HarborSync Servislerinin Kubernetes Karsiligi

Bilesen	Kubernetes nesnesi	Replica	Config/Secret	Dis bagimlilik
API Gateway	Deployment + Service + Ingress	2	JWT secret, downstream URL, Redis host	Redis, Vessel, Task
Vessel Service	Deployment + Service	2	DB/RabbitMQ secret, env config	PostgreSQL vessel_db, RabbitMQ
Telemetry Service	Deployment + Service	3	Redis/RabbitMQ env	Redis, RabbitMQ
Congestion Analysis	Deployment + Service	2	RabbitMQ env, threshold ConfigMap	RabbitMQ
Task Assignment	Deployment + Service	2	DB/RabbitMQ secret, Vessel URL	PostgreSQL task_db, RabbitMQ, Vessel
Notification Service	Deployment + Service	2	RabbitMQ env, log config	RabbitMQ, logging stack
PostgreSQL vessel_db	External DB / managed / VM	-	DB credentials	Backup/PITR
PostgreSQL task_db	External DB / managed / VM	-	DB credentials	Backup/PITR
RabbitMQ	Operator/StatefulSet veya external	3 prod	User/pass/TLS secret	PV, backup definitions
Redis	Operator/StatefulSet veya external	3 prod	Auth/TLS secret	Rate limit/cache

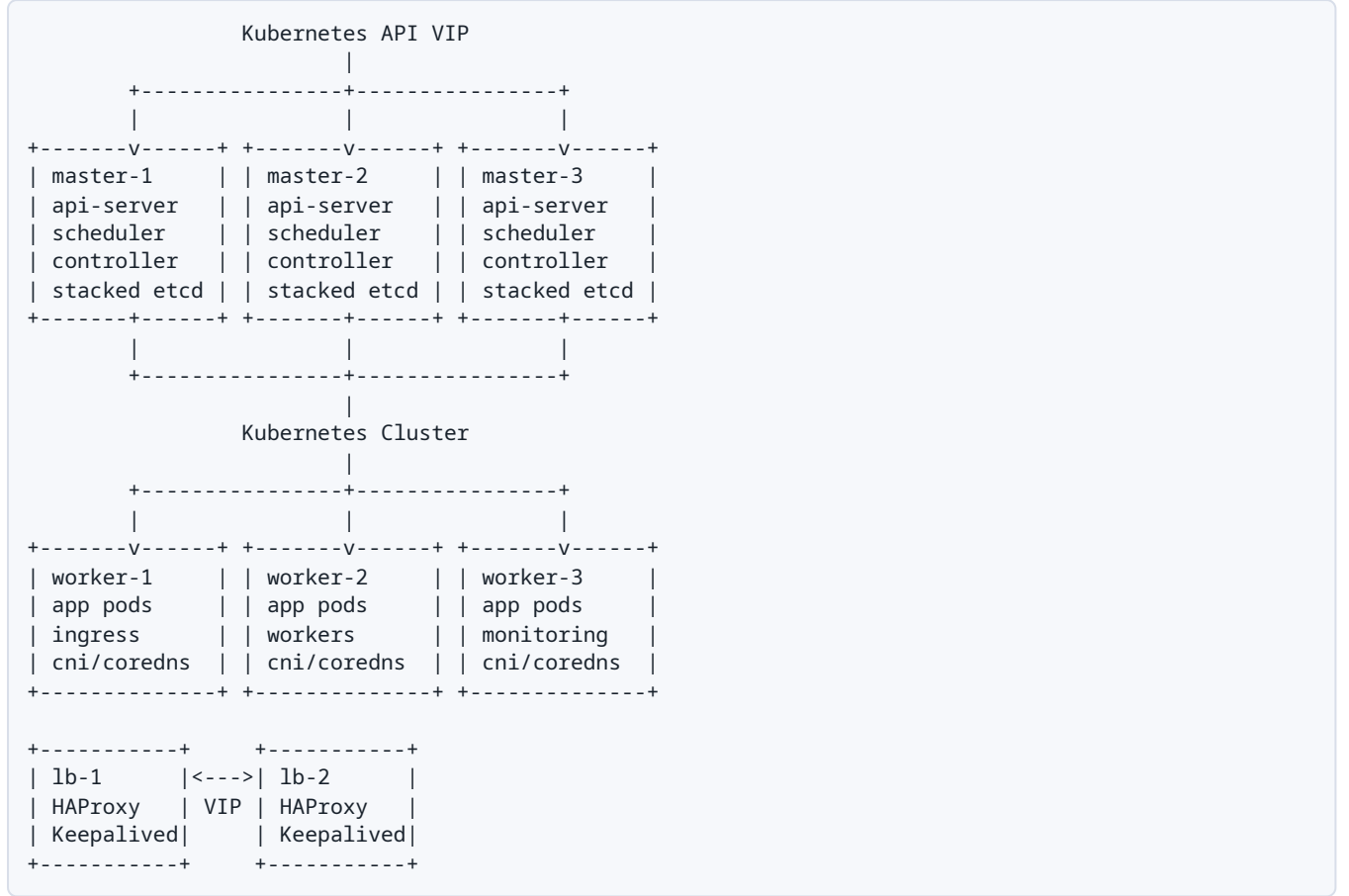
Not: Yeni baslayan Kubernetes pratigi icin PostgreSQL'in cluster disinda tutulmasi dogru yaklasimdir. Böylece önce Deployment, Service, Ingress, HPA, GitOps, monitoring ve rollout pratigi netlesir.

9.3 Hedef Multi-Master Topoloji



Şekil 3: HarborSync hedef multi-master Kubernetes topolojisi.

9.4 Cluster Node Diyagrami



Şekil 4: Multi-master cluster node ve load balancer yerlesimi.

9.5 Uygulama Kaynak Hesabi

13.3 bölümündeki ölçümlerden ve HA replica kararlarından hareketle uygulama request toplamı aşağıdaki gibidir. Bu hesap sadece HarborSync uygulama pod'larıdır; Kubernetes add-on'ları, monitoring/logging, image cache ve bos kapasite ayrıca eklenmelidir.

Servis	Replica	CPU request	RAM request	Toplam CPU	Toplam RAM
API Gateway	2	250m	512Mi	500m	1Gi
Vessel Service	2	300m	768Mi	600m	1.5Gi
Telemetry Service	3	100m	128Mi	300m	384Mi
Congestion Analysis	2	250m	512Mi	500m	1Gi
Task Assignment	2	500m	1Gi	1000m	2Gi
Notification Service	2	100m	128Mi	200m	256Mi
Toplam uygulama	13 pod	-	-	3.1 CPU	~6.1Gi RAM

Platform payı eklenmiş worker ihtiyaci:

Kalem	CPU	RAM	Not
HarborSync uygulama request	~3.1 CPU	~6.1Gi	13 pod
Ingress + CNI + CoreDNS + metrics-server + cert-manager + MetalLB	~1-2 CPU	~2-4Gi	Minimal platform
Argo CD	~0.5-1 CPU	~1-2Gi	GitOps
Prometheus + Grafana + Loki	~2-4 CPU	~6-12Gi	Retention'a göre büyür
Bosluk payi	+30-50%	+30-50%	Rolling update ve node failure için
Önerilen worker havuzu	12 CPU+	36-48Gi+	3 worker x 4 CPU / 12-16Gi

Sonuç: HarborSync uygulaması tek basına küçük/orta boydur; worker boyutunu büyüten ana kalem monitoring/logging ve DevOps add-on'larıdır. Bu nedenle 3 worker x 4 vCPU / 12-16 GB RAM mantıklı başlangıçtır.

9.6 Önerilen VM Kaynakları

VM tipi	Adet	vCPU	RAM	Disk	Not
Load Balancer	2	1	1-2 GB	20 GB	HAProxy + Keepalived
Master	3	2	6-8 GB	60-80 GB SSD	Stacked etcd; disk latency önemli
Worker	3	4	12-16 GB	100-150 GB SSD	App + add-on + monitoring
External DB	1	2-4	4-8 GB	100-200 GB SSD	Lab için tek VM; prod için HA/managed
CI Runner	1	2	4 GB	50-100 GB	Build/test/deploy job'ları
Monitoring/Logging VM	Opsiyonel	2-4	8-16 GB	100-300 GB	Cluster dışı gözlemleme istenirse

Minimum Proxmox host kapasitesi:

Seviye	CPU	RAM	Disk
Minimum lab	8 core / 16 thread	64 GB	1 TB SSD/NVMe
Rahat lab/pilot	12-16 core / 24-32 thread	128 GB	2 TB+ SSD/NVMe
Gerçek fiziksel HA	3 fiziksel Proxmox node	128 GB+ toplam/node profiline göre	Replicated/ZFS/Ceph tasarımı

Tek Proxmox host üzerinde 3 master + 3 worker kurmak Kubernetes seviyesinde HA pratigidir; fiziksel HA değildir. Host kapanırsa tüm cluster gider. Gerçek HA için en az 3 fiziksel Proxmox node gerekir.

9.7 Ortam ve Namespace Planı

Baslangıçta tüm ortamları aynı anda açmak yerine kademeli ilerlemek daha sağlıklıdır.

Faz	Namespace	Replica yaklasimi	Amaç
Faz 1	harborsync-dev	Kritik servisler 1, Gateway 1	Manifest, ingress, secret, config denemesi
Faz 2	harborsync-staging	Gateway/Vessel/Task 2, Telemetry 2	Rolling update, HPA, smoke test
Faz 3	harborsync-prod-sim	Rapordaki replica hedefleri	HA, node failure, backup/restore tatbikati

Prod benzeri senaryoda namespace'ler:

```

harborsync-dev
harborsync-staging
harborsync-prod
observability
ingress-nginx
cert-manager
argocd
external-secrets
rabbitmq-system      # RabbitMQ operator kullanilirsa
redis-system         # Redis operator kullanilirsa

```

9.8 Add-on Planı

Add-on	Gerekli mi	Kaynak etkisi	Not
CNI	Evet	Düşük/orta	Cilium veya Calico
CoreDNS	Evet	Düşük	Control plane için temel
Ingress Controller	Evet	Orta	NGINX Ingress veya Traefik
MetalLB	Bare-metal/Proxmox için evet	Düşük	LoadBalancer servis tipi için
cert-manager	Önerilir	Düşük	TLS otomasyonu
metrics-server	Evet	Düşük	HPA için gerekli
Argo CD	Önerilir	Orta	GitOps deploy modeli
Prometheus/Grafana	Önerilir	Orta/yüksek	Metrik ve dashboard
Loki/Promtail	Önerilir	Orta/yüksek disk	Log retention'a göre büyür
KEDA	Önerilir	Düşük	RabbitMQ queue depth bazlı scale
External Secrets	Önerilir	Düşük	Vault/cloud secret entegrasyonu

9.9 Ingress, Service ve Network Akisi

```

Internet / Lab Client
|
v
App VIP (HAProxy/Keepalived veya MetalLB)
|
v
Ingress Controller
|
+--> /api/vessels/**  -> api-gateway -> vessel-service
+--> /api/tasks/**   -> api-gateway -> task-assignment-service
+--> /telemetry/*     -> telemetry-service veya gateway route kararı

Cluster internal:
task-assignment-service -> vessel-service:8081
services -> rabbitmq:5672
gateway/telemetry -> redis:6379
vessel/task -> external PostgreSQL endpoint

```

Şekil 5: Ingress, service ve cluster içi network akisi.

Güvenlik kararı: Dis dünyaya sadece Ingress/Gateway açılmalı. Vessel, Task, Congestion, Notification, RabbitMQ ve Redis ClusterIP veya internal endpoint olarak kalmalı. Telemetry ingest doğrudan dis açılacaksa ayrı rate-limit/auth kararı gerekir.

9.10 Scheduling, Anti-Affinity ve PDB

Bileşen	Scheduling önerisi
Gateway	2 replica farklı worker node'lara yayılmalı
Telemetry	3 replica mümkünse 3 worker'a dağılmalı
Task Assignment	Vessel Service ile aynı node'a zorunlu bağlanmamalı; network üzerinden konuşmalı
RabbitMQ	Cluster içinde çalışacaksa 3 node, podAntiAffinity ve ayrı PV
Monitoring	Worker kaynaklarını tüketmemesi için request/limit net olmalı
PDB	Gateway, Vessel, Task, Telemetry için minimum available tanımlanmalı

9.11 HarborSync İçin Karar Özeti

Karar başlığı	Nihai öneri
Master	3 adet, 2 vCPU / 6-8 GB RAM / 60-80 GB SSD
Worker	3 adet, 4 vCPU / 12-16 GB RAM / 100-150 GB SSD
LB	2 adet, 1 vCPU / 1-2 GB RAM / 20 GB disk
DB	Kubernetes dışında 1 DB VM ile başla; prod için HA/managed PostgreSQL
RabbitMQ	Lab için cluster içi mümkün; prod için 3 node operator/managed
Redis	Lab için tek/HA Redis; prod için Sentinel/managed
Replica	Gateway 2, Vessel 2, Telemetry 3, Congestion 2, Task 2, Notification 2
Kaynak toplamı	Uygulama request ~3.1 CPU / ~6.1Gi RAM; add-on ve boşlukla 36-48Gi worker RAM hedeflenmeli
Ortam	Dev ile başla, staging ekle, prod simülasyonu en son
CI/CD	Argo CD + image registry + test/build/scan pipeline
Monitoring/logging	Prometheus, Grafana, Loki; RabbitMQ queue depth ve JVM/DB metrikleri kritik
Backup	External PostgreSQL PITR, RabbitMQ definitions/PV snapshot, Velero manifests

10. HA İhtiyacı

HA ihtiyacı yüksektir çünkü sistem operasyonel karar üretir ve event zincirindeki kopukluk task/notification gecikmesine neden olur.

Bileşen	HA seviyesi	Öneri
Kubernetes API	Kritik	3 control-plane, stacked veya external etcd; API LB
Gateway	Yüksek	En az 2 replica, PodDisruptionBudget
Telemetry	Yüksek	En az 3 replica; rate ve publish latency izlenmeli
Congestion	Yüksek	En az 2 replica; queue depth ile autoscale
Task Assignment	Kritik	En az 2 replica; idempotency iyileştirilmeli
Vessel	Orta-yüksek	En az 2 replica
Notification	Orta	En az 2 replica; aynı queue'da competing consumer semantigi dikkate alınmalı
RabbitMQ	Kritik	3 node cluster, quorum queues
PostgreSQL	Kritik	Primary + 2 replica, automatic failover
Redis	Orta-yüksek	3 node Sentinel/cluster; rate limiter için HA

11. Backup/Restore İhtiyacı

Veri	Kritik mi	Backup önerisi	RPO/RTO hedefi
vessel_db	Evet	Günlük full + WAL archiving/PITR	RPO <= 5 dk, RTO <= 30 dk
task_db	Evet	Günlük full + WAL archiving/PITR	RPO <= 5 dk, RTO <= 30 dk
RabbitMQ durable queues	Evet, kısa süreli	Definitions export + persistent volumes snapshot; quorum queue	RPO <= 5-15 dk
Redis rate/cache	Düşük/orta	Rate/cache ise backup gerekmez; kritik state eklenirse AOF/RDB	RPO best-effort
Notification logs	Audit gerekiyorsa evet	Merkezi log sistemi retention + object storage	Retention 30-180 gün
Kubernetes manifests/secrets	Evet	GitOps repo + sealed/external secrets	Git restore

Restore testleri ayda en az bir kez stage ortamında denenmeli. PostgreSQL için `pgBackRest`, `Barman` veya operator native backup; RabbitMQ için definitions + PV snapshot; Kubernetes için Velero önerilir.

12. Güvenlik İhtiyacı

Mevcut güvenlik durumu demo seviyesindedir:

- Gateway JWT filtresi sadece HS256 imzasini dogrular; exp/nbf/aud/iss/scope kontrolü yoktur (`JwtAuthFilter.java:54-65`).
- Default secret ve credential degerleri `.env.example` ve compose içinde görünür.
- Direct service portlari compose'da host'a açıktır.
- TLS/mTLS, network policy, pod security, image signing ve secret manager yoktur.

Üretim önerileri:

Alan	Öneri
Kimlik dogrulama	OIDC provider, JWT claim validation, token expiry, JWKS/key rotation
Network	Sadece Ingress -> Gateway dis açık; servisler ClusterIP; NetworkPolicy ile namespace içi kısıt
TLS	Ingress TLS, iç trafik için mTLS/service mesh opsiyonel
Secrets	Kubernetes Secret yerine External Secrets + Vault/cloud secret manager
RabbitMQ/PostgreSQL/Redis	Strong credentials, TLS, least privilege user, network isolation
Container	Non-root user, read-only root FS, seccomp, drop capabilities
Supply chain	Image scan, SBOM, cosign signing, admission policy
Observability	Prometheus/Grafana, centralized logging, tracing, alerting
API hardening	Request size limit, schema validation, WAF/rate policy, actuator exposure kısıtlama

13. CPU/RAM İhtiyaci

Kodda Kubernetes resource request/limit tanımı yoktur; aşağıdaki değerler pilot/üretime geçiş için başlangıç önerisidir. JVM servislerinde gerçek heap ölçümü sonrası ayar revize edilmelidir.

13.1 Uygulama Pod Başlangıç Kaynakları

Servis	Request CPU	Request RAM	Limit CPU	Limit RAM	Not
gateway	250m	512Mi	1000m	1Gi	Reactive gateway + Redis rate limit
vessel-service	300m	768Mi	1500m	1.5Gi	Spring + JPA
telemetry-service	200m	256Mi	1000m	512Mi	Go servis, network/Rabbit publish yoğun
congestion-analysis	250m	512Mi	1000m	1Gi	Stateless rule engine
task-assignment-service	500m	1Gi	2000m	2Gi	JPA + Rabbit consumer + WebClient
notification-service	150m	256Mi	500m	512Mi	FastAPI/aio-pika

13.2 Stateful Altyapı Başlangıç Kaynakları

Bileşen	Replica	Request CPU	Request RAM	Storage
RabbitMQ	3	1000m	2Gi	50-100Gi SSD/PV per node
PostgreSQL vessel	3	1000m	4Gi	100Gi+ SSD/PV
PostgreSQL task	3	1000m	4Gi	100Gi+ SSD/PV
Redis	3	500m	1Gi	10-20Gi opsiyonel

13.3 Çalıştırma Bazlı Kaynak Ölçümü ve Net Request/Limit Tablosu

Bu bölüm 2026-06-25 tarihinde proje Docker Compose ile çalıştırılarak hazırlandı. Host üzerinde keycloak container'ini 127.0.0.1:8080 portunu kullandığı için Compose Gateway 8080'e bind edemedi; mevcut servise dokunmadan aynı Gateway imajı `api-gateway-measure` adıyla `harborsync_default` network'ünde `18080:8080` portuyla çalıştırıldı.

Ölçüm yöntemi:

- Tüm servis health endpointleri kontrol edildi: Gateway, Vessel, Telemetry, Congestion, Task Assignment ve Notification UP döndü.
- Boşta `docker stats --no-stream` ile kaynak tüketimi alındı.
- Kısa örnek yük uygulandı: Gateway üzerinden Vessel API'ye 80 GET, Telemetry Service'e 160 POST gönderildi. Bu trafik Congestion -> Task Assignment -> Notification event zincirini tetikledi.
- Yük sonrası `task_db` içinde 92 adet `PENDING` task oluştugu görüldü.
- RabbitMQ tarafında `telemetry.processed`, `congestion.alert.task-assignment`, `congestion.alert.notification`, `task.created` kuyrukları bosaldı. Buna karşılık consumer'i olmayan `congestion.alert` kuyruğunda 92 mesaj birikti; bu, önceki bottleneck notunu çalıştırma ile de doğruladı.

Gözlenen runtime değerleri:

Bileşen	Boşta RAM	Yükte tepe CPU	Yükte tepe RAM	Sınıflandırma
API Gateway	243Mi	17.20%	259Mi	Basit/orta API gateway
Vessel Service	287Mi	23.94%	297Mi	Basit API + JPA
Telemetry Service	5Mi	2.83%	7Mi	Hafif ingest API/producer
Congestion Analysis	195Mi	9.85%	197Mi	Worker/consumer
Task Assignment	415Mi	63.52%	484Mi	Yogun worker/API + JPA
Notification Service	41Mi	2.18%	41Mi	Hafif worker/consumer
RabbitMQ	151Mi	81.32%	152Mi	Mesaj broker
Redis	5Mi	2.96%	5Mi	Küçük Redis/cache
PostgreSQL vessel_db	65Mi	5.57%	66Mi	Küçük lab DB
PostgreSQL task_db	52Mi	1.16%	52Mi	Küçük lab DB

Net Kubernetes request/limit önerisi:

Servis tipi / bileşen	CPU request	RAM request	CPU limit	RAM limit
API Gateway	250m	512Mi	1000m	1Gi
Vessel Service	300m	768Mi	1500m	1536Mi
Telemetry Service	100m	128Mi	500m	512Mi
Congestion Analysis	250m	512Mi	1000m	1Gi
Task Assignment Service	500m	1Gi	2000m	2Gi
Notification Service	100m	128Mi	500m	512Mi
RabbitMQ küçük lab	500m	512Mi	2000m	2Gi
RabbitMQ production baseline	1000m	2Gi	2000m	4Gi
Redis küçük lab	100m	128Mi	500m	512Mi
Redis production baseline	250m	512Mi	500m	1Gi
PostgreSQL vessel_db küçük lab	500m	512Mi	2000m	2Gi
PostgreSQL task_db küçük lab	500m	512Mi	2000m	2Gi
PostgreSQL production baseline	1000m	2Gi	2000m	4Gi

Yorum:

- Docker CPU yüzdesi kısa burst ölçümüdür; uzun süreli kapasite testi yerine başlangıç request/limit belirlemek için kullanıldı.
- Spring/JVM servislerinde heap ve native memory payı nedeniyle observed RAM'in en az 2x'i request/limit kararında korunmalıdır. Bu yüzden 195-484Mi arası gözlenen JVM pod'larına 512Mi-1Gi request önerildi.
- Task Assignment ölçümde en yüksek uygulama CPU'sunu gördü. Bunun sebebi alert tüketimi sırasında hem PostgreSQL yazması hem Vessel Service HTTP çağrısı hem RabbitMQ publish yapmasıdır.
- RabbitMQ kısa burst'te en yüksek CPU spike'i yapan altyapi bileşeni oldu. Production'da queue depth ve disk I/O metrikleri ile HPA/KEDA değil, broker cluster kapasitesi ve quorum queue tasarımı

belirleyici olacaktır.

- Telemetry Service Go ile yazıldığı için RAM footprint çok düşük; buna rağmen network/RabbitMQ publish burst'leri için 100m/128Mi altına inilmemelidir.
- PostgreSQL lab ölçümünde RAM düşük görünür; gerçek üretim için buffer/cache, connection sayısı, WAL ve index büyüklüğü nedeniyle 2Gi altına inilmemelidir.

13.4 Cluster Kapasite Önerisi

Minimum pilot: 3 worker x 8 vCPU/32 GB RAM. Stateful workload cluster içinde çalışacaksa bu kaynak ancak başlangıçtır; production için 3-5 worker x 16 vCPU/64 GB RAM daha rahat olur. Managed DB/Rabbit/Redis kullanılırsa 3 worker x 8 vCPU/32 GB RAM uygulama tarafı için yeterli başlangıçtır.

14. Replica İhtiyacı

Servis	Min replica	Autoscale önerisi	Gerekçe
gateway	2	CPU/RPS/p95 latency	Edge HA
vessel-service	2	CPU/latency	REST ve Task dependency HA
telemetry-service	3	CPU/RPS/Rabbit publish latency	En yoğun ingest noktası
congestion-analysis	2	RabbitMQ <code>telemetry.processed</code> queue depth	Stateless consumer
task-assignment-service	2	RabbitMQ <code>congestion.alert.task-assignment</code> depth + CPU	DB write ve Vessel dependency
notification-service	2	Queue depth/CPU	Bildirim gecikmesini azaltma
RabbitMQ	3	N/A	Quorum/cluster HA
PostgreSQL cluster'ları	3	N/A	Primary + replicas/failover
Redis	3	N/A	Sentinel/cluster HA

KEDA, RabbitMQ queue depth'e göre Congestion ve Task consumer'larını ölçmek için uygundur. HPA yalnız CPU ile kullanılırsa event backlog geç fark edilebilir.

15. Operasyonel Gözlemler ve İyileştirme Listesi

- `congestion.alert` consumer'siz queue birikimi düzeltilmeli. Congestion Analysis producer routing key kullanmalı; kendi adına consume edilmeyen durable queue declare etmemeli.
- Task Assignment idempotency güçlendirilmeli. Aynı alert tekrar teslim edilirse duplicate task üretimi mümkün; correlationId + sector + alertType için idempotency key düşünülebilir.
- RabbitMQ publisher confirm kullanımı eklenmeli. Şu an publish çağrıları uygulama seviyesinde başarı kabul ediyor; broker confirm/return handling üretim için kritik.
- Notification log dosyası container local filesystem yerine stdout + merkezi logging olmalı.
- JWT claim validation ve secret rotasyonu eklenmeli.
- Kubernetes manifests/Helm chart, resource request/limit, readiness/liveness probe, PDB ve NetworkPolicy yazılmalı.
- DB migration deployment süreci netleşmeli: Flyway otomatik kosuyor, fakat multi-replica deployment sırasında migration lock ve rollout sırası kontrol edilmeli.
- Observability eklenmeli: RabbitMQ queue depth, consumer lag, DB latency, HTTP p95, error rate, JVM heap, Go runtime metrics.

16. Sonuç

HarborSync mikroservis sinirlari net, event-driven akisi anlasilir ve Docker Compose ile dogrulanmis bir demo platformudur. Eksiksiz uygulama servis sayisi 6'dir. Üretim veya ciddi pilot hedefi için esas is, uygulama kodundan çok operasyonel katmandadir: HA RabbitMQ/PostgreSQL/Redis, Kubernetes deployment standartlari, CI/CD, secret yönetimi, observability, backup/restore ve güvenlik sertlestirmesi.

Multi-master Kubernetes için önerilen baslangıç: 3 control-plane + 3 worker, uygulama pod'lari Deployment olarak en az 2 replica, Telemetry 3 replica, RabbitMQ/PostgreSQL/Redis HA olarak operator/managed servislerle kurulmalidir. Sistemin en kritik performans eksenini Telemetry -> RabbitMQ -> Congestion -> Task Assignment zinciridir; kapasite testleri öncelikle bu zincir üzerinde yapılmalıdır.

17. Proxmox İhtiyacı - Demo Ortamı

Bu bölüm demo ve pratik ortamı içindir. Sisteme gerçek üretim trafiği veya büyük veri girmeyecek; maksimum test verisi, kısa süreli load testleri ve Kubernetes/DevOps pratiği yapılacaktır. Bu nedenle Proxmox tarafında aşırı kaynak istemeye gerek yoktur. Gerçek production bir sistem olsaydı 9. bölümdeki daha geniş kaynak yaklaşımı, HA DB, daha büyük worker kaynakları, ayrı monitoring kapasitesi ve fiziksel HA dikkate alınmalıydı.

17.1 Demo Varsayımı

Baslik	Demo kararı
Trafik	Düşük/orta test trafiği, kısa süreli burst
Veri	Az miktarda test verisi
Database	Kubernetes dışında tek PostgreSQL VM yeterli
RabbitMQ/Redis	Demo için cluster içinde küçük kaynakla çalışabilir veya tek VM/container olarak tutulabilir
Monitoring	Hafif Prometheus/Grafana/Loki; uzun retention yok
CI/CD	Küçük bir runner VM yeterli
Fiziksel HA	Tek Proxmox host üzerinde Kubernetes HA pratiği; gerçek fiziksel HA değil

17.2 İstenen Demo VM Kaynakları

VM	Adet	vCPU	RAM	Disk	Açıklama
Load Balancer	2	1	1 GB	20 GB	HAProxy + Keepalived için yeterli
Master	3	2	4-6 GB	50-60 GB SSD	Stacked etcd demo için yeterli; mümkünse SSD
Worker	3	4	8-12 GB	80-100 GB SSD	Uygulama pod'ları + temel add-on'lar
External DB	1	2	4 GB	80-100 GB SSD	Az test verisi için PostgreSQL yeterli
CI Runner	1	2	4 GB	50 GB	Build/test/deploy işleri için
Monitoring VM	Opsiyonel	2	4-8 GB	80-100 GB	Monitoring cluster dışında tutulacaksa

17.3 Minimum Demo Toplamı

Monitoring ayrı VM olarak açılmaz, monitoring/logging cluster içinde hafif tutulursa yaklaşık ihtiyaç:

Kaynak	Yaklaşık toplam
vCPU	25 vCPU
RAM	50-65 GB
VM disk	650-800 GB

Bu toplam, VM'lere ayrılan teorik kaynaktır. Proxmox host tarafında hypervisor, disk snapshot, image cache ve büyümeye payı için ek boşluk bırakılmalıdır.

17.4 Rahat Demo Toplamı

Monitoring VM de açılacaksa veya worker'lara 12 GB RAM verilecekse daha rahat demo ihtiyacı:

Kaynak	Yaklaşık toplam
vCPU	27-30 vCPU
RAM	70-85 GB
VM disk	800 GB - 1 TB

17.5 Proxmox Fiziksel Host Önerisi

Seviye	CPU	RAM	Disk	Uygunluk
Minimum demo	8 core / 16 thread	64 GB	1 TB SSD/NVMe	Çalışır; monitoring/logging hafif tutulmalı
Rahat demo	12 core / 24 thread	96 GB	1.5 TB SSD/NVMe	Daha sorunsuz demo ve snapshot alanı
Çok rahat demo	12-16 core / 24-32 thread	128 GB	2 TB SSD/NVMe	Daha fazla ortam ve monitoring retention için

Demo için en makul istek: 8 core / 16 thread CPU, 64 GB RAM ve 1 TB SSD/NVMe ile başlanabilir. Eğer elde imkan varsa 96 GB RAM ve 1.5 TB disk daha rahat olur. 128 GB RAM ve 2 TB disk production benzeri lab için konfor sağlar ama bu demo için zorunlu değildir.

17.6 Demo İçin Kaynakları Küçük Tutma Kararları

Alan	Demo kararı	Gerçek sistemde ne olurdu?
PostgreSQL	Tek external DB VM	HA PostgreSQL, replica, PITR, ayrı backup storage
RabbitMQ	Küçük cluster içi kurulum veya tek node lab	3 node quorum queue cluster veya managed RabbitMQ
Redis	Tek küçük Redis veya hafif HA	Sentinel/managed Redis
Monitoring retention	Kısa süreli, düşük disk	Uzun retention, ayrı disk ve kapasite planı
Worker RAM	8-12 GB	16 GB+ ve node sayısı artırımı
Disk	80-100 GB worker disk	Image cache, log, volume ve retention'a göre 150-300 GB+
Fiziksel HA	Tek Proxmox host kabul	En az 3 fiziksel host, shared/replicated storage

17.7 Net Talep Metni

Demo ortamı için Proxmox tarafında istenecek kaynak özeti:

```
2 adet Load Balancer VM: 1 vCPU, 1 GB RAM, 20 GB disk
3 adet Master VM:      2 vCPU, 4-6 GB RAM, 50-60 GB SSD disk
3 adet Worker VM:      4 vCPU, 8-12 GB RAM, 80-100 GB SSD disk
1 adet External DB VM:  2 vCPU, 4 GB RAM, 80-100 GB SSD disk
1 adet CI Runner VM:   2 vCPU, 4 GB RAM, 50 GB disk
Opsiyonel Monitoring VM: 2 vCPU, 4-8 GB RAM, 80-100 GB disk
```

Bu kaynaklar HarborSync demosu, Kubernetes multi-master pratiği, rolling update, temel monitoring, GitOps ve kısa süreli test yükleri için yeterlidir. Gerçek production hedeflenseydi daha yüksek worker kaynakları, HA database, daha büyük disk, uzun log/metric retention ve fiziksel Proxmox HA tasarımı gerekir.

18. Uygulama Fazi - Güncel Mentor Kararına Göre Yol Haritası

Bu bölüm raporun önceki analizinden sonra alınan güncel uygulama kararlarını temel alır. Önceki bölümlerde dış DB ve ayrı LB VM seçenekleri değerlendirilmişti; mentor kararıyla demo/pratik ortamı için tasarım artık şu şekilde uygulanacaktır:

Konu	Güncel karar
Master node	3 adet master/control-plane
Worker node	3 adet worker
Load balancer	Ayrı LB VM yok; HAProxy + Keepalived master node'lara kurulacak
Database	PostgreSQL Kubernetes içine kurulacak
RabbitMQ	Kubernetes içine kurulacak
Redis	Kubernetes içine kurulacak
Monitoring	Helm ile Kubernetes içine kurulacak
CI runner	Şimdilik iptal; manuel build/deploy veya lokal pipeline ile ilerlenebilir
Amaç	Demo, öğrenme, mikroservis deploy pratiği, HA davranışı gözlemlleme

Bu kararlar cluster daha öğretici hale gelir; çünkü sadece uygulama pod'larını değil, stateful workload, storage, Helm release, monitoring ve node failure etkilerini de göreceğiz. Bedeli şudur: Storage, backup, StatefulSet/Operator ve kaynak yönetimi artık daha dikkatli yapılmalıdır.

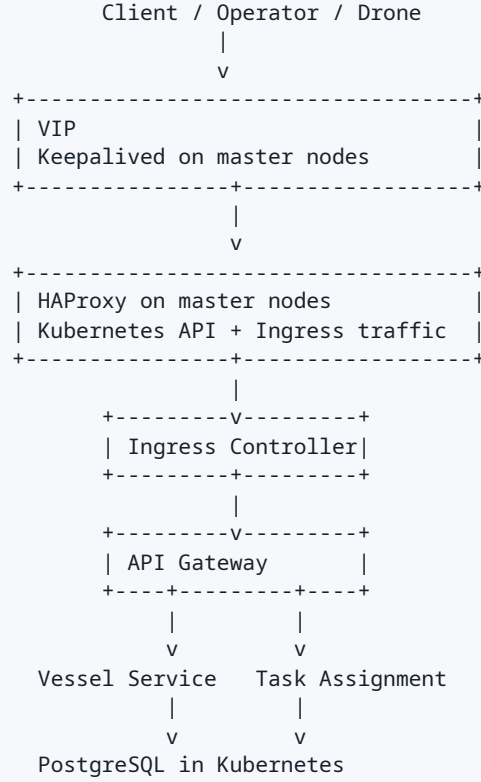
18.1 Bu Projede Liderlik Prensibi

Bu uygulamayı yaparken amacımız komut ezberlemek değil, her kararın neden verildiğini anlayarak ilerlemek. Her adımda üç soruya cevap arayacağız:

- Bu bileşen cluster içinde hangi problemi çözüyor?
- Bu bileşen bozulursa hangi servis etkilenir?
- Kurulumdan sonra çalıştığını hangi somut sinyalle doğrularız?

Örnek: PostgreSQL'i kurmak sadece `helm install` çalıştırmak değildir. Önce şunu bilmeliyiz: Vessel Service ve Task Assignment kalıcı veri tutar; bu yüzden PostgreSQL pod'u silinse bile veri PV üzerinde kalmalıdır. Doğrulama da sadece pod `Running` görmek değildir; database'e bağlanıp schema migration'ın oluştuğunu ve servislerin health verdiğini görmektir.

18.2 Güncel Hedef Mimari



Telemetry Service -> Redis in Kubernetes

Telemetry Service -> RabbitMQ in Kubernetes -> Congestion -> Task/Notification

Monitoring Helm chart -> Prometheus/Grafana/Loki inside Kubernetes

Şekil 6: Güncel mentor kararına göre HarborSync Kubernetes hedef mimarisi.

18.3 Node Roller

Node	Rol	Üzerinde olacak ana işler
master-1	Control-plane + HAProxy/Keepalived	API server, scheduler, controller-manager, etcd, LB süreci
master-2	Control-plane + HAProxy/Keepalived	API server, scheduler, controller-manager, etcd, LB süreci
master-3	Control-plane + HAProxy/Keepalived	API server, scheduler, controller-manager, etcd, LB süreci
worker-1	Worker	Uygulama pod'ları, ingress, monitoring parçaları
worker-2	Worker	Uygulama pod'ları, RabbitMQ/Redis/PostgreSQL pod'ları
worker-3	Worker	Uygulama pod'ları, monitoring/logging, stateful replica dağılımı

Master node'lara LB kurulacak olması şu anlama gelir: Ayrı LB VM yönetmeyeceğiz, fakat master node'ların işletim sistemi seviyesinde HAProxy/Keepalived süreçleri doğru yönetilmelidir. Bu süreçler Kubernetes pod'u gibi değil, node üzerindeki systemd servisleri gibi düşünülmelidir.

18.4 Uygulama Sırası

Bu sırayı bozmayacağız; çünkü alttaki katman doğrulanmadan üst katmana çıkmak hata ayıklamayı zorlaştırır.

Sıra	Faz	Amaç	Bittiğini nasıl anlarız?
1	VM ve OS hazırlığı	3 master + 3 worker hazır olmalı	Hostname, static IP, DNS/hosts, SSH, disk hazır
2	HAProxy + Keepalived	Kubernetes API VIP hazır olmalı	VIP hangi master aktifse orada görülür, API portu cevap verir
3	Kubernetes bootstrap	Multi-master control-plane kurulmalı	<code>kubect1 get nodes</code> 6 node'u gösterir
4	CNI kurulumu	Pod network çalışmalı	CoreDNS Running, pod'lar node'lar arası konuşur
5	StorageClass	Stateful workload için PV üretimi	Test PVC Bound olur
6	Ingress controller	HTTP giriş katmanı hazır olmalı	Ingress controller pod'ları Running, test route cevap verir
7	cert-manager/MetalLB gerekiyorsa	TLS ve LoadBalancer davranışı	Certificate/LoadBalancer kaynakları hazır
8	PostgreSQL	Vessel ve Task DB hazır	DB pod Running, PVC Bound, connection test başarılı
9	RabbitMQ	Event broker hazır	Management/AMQP erişimi ve queue declare başarılı
10	Redis	Gateway rate-limit ve telemetry cache hazır	Redis ping başarılı
11	Monitoring Helm	Metrik/log görünürlüğü	Grafana/Prometheus açılır, node/pod metrikleri gelir
12	HarborSync manifests	Uygulamalar deploy edilir	Tüm health endpointleri UP
13	Demo flow	Uçtan uca senaryo çalışır	Telemetry -> alert -> task -> notification akışı görülür
14	Failure test	HA davranışı anlaşılır	Pod silme/node drain sonrası servis toparlanır

18.5 Kubernetes İçindeki Stateful Bileşenler

Bu kararda PostgreSQL, RabbitMQ ve Redis Kubernetes içinde olacağı için önce storage mantığını netleştirmeliyiz.

Bileşen	Kubernetes tipi	Demo yaklaşımı	Dikkat edilecek nokta
PostgreSQL	StatefulSet veya Helm chart	Tek primary ile başlanabilir	PVC silinirse veri gider; migration sırası önemli
RabbitMQ	Helm chart/operator	Demo için tek node veya 3 replica	Queue durability ve disk alanı izlenmeli
Redis	Helm chart	Tek node veya sentinel opsiyonel	Gateway rate-limit Redis'e bağlı
Monitoring	Helm chart	kube-prometheus-stack + opsiyonel Loki	Disk retention küçük tutulmalı

Demo olduğu için her stateful bileşeni en baştan production HA ile kurmak zorunda değiliz. Ama şunu bilerek ilerleyeceğiz: PostgreSQL ve RabbitMQ cluster içindeyse worker diskleri ve StorageClass artık uygulamanın güvenilirliğinin parçasıdır.

18.6 Namespace Planı

Başlangıçta fazla namespace açmayacağız. Önce sistemi çalıştırıp sonra ayıracağız.

Namespace	İçerik
harborsync	Uygulama servisleri: gateway, vessel, telemetry, congestion, task, notification
data	PostgreSQL, Redis, RabbitMQ
monitoring	Prometheus, Grafana, Loki/Promtail
ingress-nginx	Ingress controller
cert-manager	TLS otomasyonu gerekiyorsa

Bu ayırım hata ayıklamada yardımcı olur. Örneğin uygulama çalışmıyorsa `harborsync`; database sorunu varsa `data`; metrik/log sorunu varsa `monitoring` namespace'ine bakarız.

18.7 Manifest ve Helm Yaklaşımı

HarborSync servisleri için kendi manifestlerimizi yazacağız. PostgreSQL, RabbitMQ, Redis ve monitoring için Helm chart kullanmak daha mantıklı; çünkü bu bileşenlerin StatefulSet, Service, Secret, PVC ve probe ayarları elle yazıldığında hata riski artar.

Bileşen	Yaklaşım
HarborSync uygulama servisleri	Kendi Deployment/Service/ConfigMap/Secret manifestleri
PostgreSQL	Helm chart veya operator
RabbitMQ	Helm chart veya operator
Redis	Helm chart
Monitoring	Helm chart, tercihen kube-prometheus-stack
Loki	Helm chart, retention düşük
Ingress	Helm chart

Burada seni ezber komutlara boğmayacağız. Her Helm release için önce values dosyasını okuyacağız, sonra sadece gerekli ayarları override edeceğiz. Mantık şu: Chart'ın varsayılanını anlamadan values yazmak kör deploy yapmaktır.

18.8 HarborSync Deploy Sırası

Uygulama servislerini de sırayla kuracağız. Çünkü bağımlılık zinciri var.

Sıra	Servis	Ön koşul	Doğrulama
1	PostgreSQL vessel_db/task_db	StorageClass hazır	DB connection ve migration
2	RabbitMQ	data namespace hazır	Exchange/queue oluşumu
3	Redis	data namespace hazır	PING/PONG
4	Vessel Service	vessel_db + RabbitMQ	/actuator/health UP
5	Task Assignment	task_db + RabbitMQ + Vessel URL	/actuator/health UP
6	Telemetry Service	Redis + RabbitMQ	/health UP
7	Congestion Analysis	RabbitMQ	/actuator/health UP
8	Notification Service	RabbitMQ	/health UP ve RabbitMQ connected
9	API Gateway	Redis + Vessel + Task	/actuator/health UP ve route testi
10	Ingress	Gateway service	Dışarıdan API erişimi

Özellikle API Gateway'i en sona bırakıyoruz. Çünkü Gateway dış kapıdır; arkadaki servisler sağlıklı değilken Gateway test etmek yanıltıcı olur.

18.9 İlk Basit Basari Kriteri

İlk hedefimiz şudur:

```
kubectl get nodes
-> 3 master + 3 worker Ready

kubectl get pods -A
-> system pod'ları Running
-> data namespace pod'ları Running
-> harborsync namespace pod'ları Running

Demo akışı:
1. Vessel oluştur
2. Telemetry gönder
3. Congestion alert üretildiğini gör
4. Task oluştuğunu gör
5. Notification log/metrik çıktısını gör
```

Şekil 7: Uygulama fazı için ilk başarı kriteri.

Bu noktaya gelmeden HPA, TLS, advanced monitoring, backup gibi konulara geçmeyeceğiz. Önce çalışan sade sistemi kuracağız; sonra olgunlaştıracacağız.

18.10 Senden Beklediğim Çalışma Şekli

Bu projede senden beklenen şey komut kopyalamak değil, her çıktıdan anlam çıkarmak olacak. Beraber ilerlerken her adımda şu bilgileri not edeceğiz:

Bakılacak şey	Neden önemli?
Pod hangi node'a schedule oldu?	Anti-affinity ve kaynak dağılımını anlamak için
Pod neden restart etti?	Probe, config, secret veya dependency hatasını ayırmak için
Service endpoint var mı?	Label selector doğru mu anlamak için
PVC Bound mu?	Stateful workload gerçekten storage aldı mı görmek için
ConfigMap/Secret doğru mu?	Uygulama environment hatalarını azaltmak için
Loglarda ilk hata ne?	Belirti ile kök sebebi ayırmak için
Health endpoint ne diyor?	Uygulama gerçekten hazır mı anlamak için

Bir hata çıktığında ilk refleksimiz tekrar deploy etmek olmayacak. Önce şu sırayla bakacağız:

1. pod status
2. pod events
3. container logs
4. env/config/secret
5. service endpoints
6. dependency health
7. resource pressure

Bu alışkanlık seni ezberden çıkarıp gerçek DevOps problem çözme seviyesine taşır.

18.11 Bu Fazda Ertelediklerimiz

Şimdilik bazı konuları bilinçli erteliyoruz. Bu zayıflık değil, doğru sıralama.

Ertelenen konu	Neden şimdi değil?
CI runner	Mentor kararıyla şimdilik iptal; önce cluster ve deploy öğrenilecek
Full production DB HA	Demo veri az; önce StatefulSet/PVC mantığı öğrenilecek
Uzun log retention	Disk tüketimini büyütür; demo için kısa retention yeterli
Service mesh	Önce temel network/service/ingress öğrenilmeli
Advanced autoscaling	Önce sabit replica ile stabil sistem kurulmalı
Gerçek fiziksel HA	Tek Proxmox üzerinde Kubernetes HA pratiği yapılacak

18.12 Benim Sana Lead Edeceğim Uygulama Sırası

Bundan sonraki pratik çalışma sırası şu olacak:

- VM planını kesinleştiririz: IP, hostname, CPU/RAM/disk.
- Master node'larda HAProxy/Keepalived tasarımı netleştiririz.
- Kubernetes cluster bootstrap sırasını belirleriz.
- CNI ve StorageClass seçimini yaparız.
- Helm ile ingress, data katmanı ve monitoring'i kurarız.
- HarborSync için image, manifest, secret ve config dosyalarını hazırlarız.
- Uygulamaları sırayla deploy ederiz.
- Demo akışını çalıştırırız.
- Pod silme, node drain, service failover testleri yaparız.
- En son rapordaki tasarım ile gerçek gözlemleri karşılaştırırız.

Bu sırada her adımda senden sadece komut çalıştırmanı değil, çıktıyı yorumlamayı isteyeceğim. Örneğin `kubect1 get pods` çıktısını gördüğümüzde soru şu olacak: "Hangi pod hazır değil, neden hazır değil, hangi dependency eksik olabilir?" Bu şekilde ilerlersek proje sonunda sadece çalışan bir cluster değil, gerçekten anladığın bir Kubernetes pratiği çıkmış olur.